

CS 64 HANDOUT #2

Introduction to MIPS Assembly Programming

Prof. Ziad Matni

Read this handout **before** coming to class. In lecture, I will expand a lot on the concepts in this document, and give lots of examples. So, these handouts are a good explanation of what I'll be talking about in class, but they are not good substitutes for missing the lecture! 😊

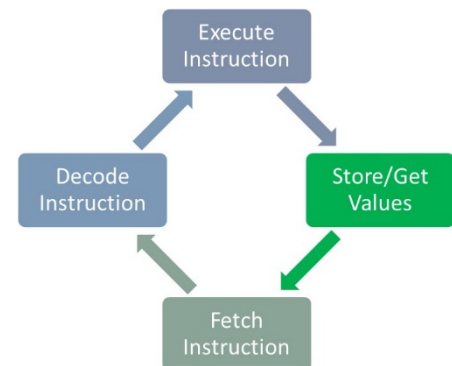
In this reading, we will discuss and clarify:

- The CPU
- Simple Addition Example
- Introduction to Basic MIPS Instructions
- Emulating MIPS Assembly Code
- Emulating a System Call
- More MIPS instructions with Additional Examples
- Pseudoinstructions
- Branching and Program Flow in MIPS Assembly

1. The Central Processing Unit (CPU)

Often referred to as the “brains” of a computer system, the CPU does all the calculations and memory storage input and output. The typical capabilities of a CPU include addition, subtraction, multiplication, division, using logic to branch from one instruction to another, reading/writing data and moving it from one location to another. You can do just about anything with a computer with only these simple instructions!

CPUs operate on the Fetch-Execute Cycle, illustrated here. The CPU starts by fetching an instruction, then figuring out what it is asking the CPU to do (aka decoding it). The CPU then executes the instruction and (optionally) interfaces with memory and stores or reads values from it. It then repeats the cycle by fetching the next instruction.



CPU instructions are made up of 1s and 0s – we call this **machine language (ML)**. An example of 32 bits of machine language instruction is: **00000001010010010100100000100000**, better shown as a hexadecimal: **0x01494820** (note this is an 8-digit hexadecimal, with each hex representing 4 bits as usual). In a MIPS CPU, this is actually an instruction to add two integers together.

Assembly language is one step of abstraction “above” ML. Here, the instructions are given *mnemonic codes* but still displayed one step at a time, fully equivalent to machine language. That is, each line of assembly language is “translatable” (we say “assembled”) into 32 bits of

machine language. The obvious advantage to using assembly language over machine language when programming a CPU directly is better human readability of the code.

Assembly language is typically generated from a high-level programming language. For example, when you write C++ code, the compiler will compile the C++ into assembly (the end result will vary according to what CPU your computer is using). The C++ compiler can also assemble your assembly code to machine language and produce an “executable” file that contains that data.

Here’s a short example of some assembly programming:

```
addi $t0, $zero, 5    # Assign 5 to register “$t0” aka “x”
addi $t1, $zero, 7    # Assign 7 to register “$t1” aka “y”
add $t2, $t1, $t0     # Add “x” to “y” and put the result in reg. “$t2” aka “z”
```

\$t0, **\$t1**, and **\$t2** are called “Registers” (there are several of those in a CPU). In a MIPS CPU, each register can contain a 32-bit value and there are 32 such registers. Most registers can be written to and read from. Registers aren’t exactly “memory” – they are temporary placeholders for values that the CPU might use in the course of executing a program.

In the example given above, **\$zero** is also a “Register”. It’s a specialized one, though, as it holds a zero value (in 32 bits, naturally) and cannot be written to (we can only read from it). There are other specialized Registers.

“**add**” and “**addi**” are **core instructions** of the MIPS CPU. There are several core instructions in MIPS and they are classified in 3 groups: Register-types, Immediate-types, and Jump-types. Each group has its own syntax, but ALL core instructions are exactly 32 bits long. For example, the 1st instruction in the example (**addi \$t0, \$zero, 5**) assembles directly to: **0010000000001000000000000000101** (or **0x20080005**). This may have started in a C++ program as **int t0 = 5**.

Since we don’t always have an actual, physical MIPS CPU handy, we will be **emulating** this code on a computer, that is, we can “pretend” we’re running these instructions on an actual MIPS CPU and “see” what the CPU would do with them.

In our CMPSC 64 course, we will be using the **spim** program – a MIPS Emulator – to run and evaluate the assembly code that we’ll be writing. The **spim** emulator is found on the CSIL computers and your TAs will demonstrate how to use it via remote login (using **ssh**) in the next lab session.

2. Simple Addition Example

The following code illustrates how to do a simple addition of $5 + 7$ and place the answer (12) in a MIPS register – as a 32-bit value.

```
addi $t0, $zero, 5      # Assign 5 to register $t0, i.e. $t0 = 0x00000005
addi $t1, $zero, 7      # Assign 7 to register $t1, i.e. $t1 = 0x00000007
add $t2, $t1, $t0
# Add the values of registers $t0 and $t1 and put the result in register $t2
```

Analysis:

Look at the 1st line of code: Note that putting the value **5** into register **\$t0** actually required us to first add 5 to what's in register **\$zero** (which holds the value 0) *and then* place the result in register \$t0. Similarly, that's how the 2nd line assigns 7 to register \$t1. That seems a little *contrived!!*

Also note that these first 2 lines had us use the instruction “**addi**” – that's MIPS for “**add immediate**”, which allows the CPU to add what's inside a register (in our case, register **\$zero**) to an “immediate” value (5 in the 1st line; 7 in the 2nd line). It so happens that the register **\$zero** is a “**special MIPS register**” that contains the value **0** *hard-wired* into it (in other words, you can read what's in register **\$zero**, but you can't write to it, i.e., you cannot change it!)

There is another way to write these first 2 lines – and that's using *pseudo-instructions*. The code would look this way:

```
li $t0, 5               # Assign 5 to register $t0, i.e. $t0 = 0x00000005
li $t1, 7               # Assign 7 to register $t1, i.e. $t1 = 0x00000007
add $t2, $t1, $t0
# Add the values of registers $t0 and $t1 and put the result in register $t2
```

The instruction “**li**” (short for “**load immediate**”) is a macro (i.e., like a shortcut) that makes it a little easier on the assembly programmer to simply assign a 32-bit value to a register. BUT TAKE NOTE: “**li**” is not a real core MIPS instruction – it's a substitute for 2 MIPS instructions (that we won't get into right now but will leave for a little later).

3. MIPS instructions... to start with...!

We've learned about 2 “**core**” MIPS instructions: **addi** and **add**.

addi is a type of MIPS instruction called an “**I-Type**”, while **add** is a type of MIPS instruction called an “**R-Type**”. We'll discuss the different types later.

Let's take a closer look at the syntax and use of these 2 core MIPS instructions:

```
addi <register1>, <register2>, <immediate value>
    register1 and register2 are register names
    immediate value is a 16-bit signed integer value
```

Example 1A: `addi $t0, $zero, 5`

Example 1B: `addi $a0, $a1, -100`
Add the value inside register \$a1 to -100
and put the result in register \$a0
i.e. equivalent of C++: `a0 = a1 + (-100);`

```
add <register_d>, <register_s>, <register_t>
    register_d is a register name (destination)
    register_s and register_t are also register names (source and target)
```

Example 2A: `add $t3, $t0, $t1`
Add the value inside register \$t0 to \$t1
and put the result in register \$t3
i.e. equivalent of C++: `t3 = t0 + t1;`

Example 2B: `add $s0, $s0, $s0`
Add the value inside register \$s0 to \$s0 (i.e. to itself)
and put the result back in register \$s0
i.e. equivalent of C++: `s0 = s0 + s0;`

4. Emulating MIPS Assembly Code

We are not actually running code in this class on an actual MIPS CPU hardware. Instead, we will be emulating (i.e. simulating what the hardware would do with the code inside a software program) the code on a program called **spim**.

So, for our simple addition code, we'll need to add more lines of code to it to tell the emulator what to do! The FULL program that can run on **spim** will actually look like this:

```
# 1. SET UP THE ENVIRONMENT FOR THE EMULATOR
.text

# 2. WRITE THE ACTUAL INSTRUCTIONS
main:
    li $t0, 5      # Assign 5 to "$t0"
    li $t1, 7      # Assign 7 to "$t1"
    add $t2, $t1, $t0 # Add "and put the result in "$t2"

# 3. SHOW THE RESULT BY EMULATING THE EXCHANGE BETWEEN CPU and OS.
# That is, print the result on the screen
# (equivalent to C++ instruction:  cout << t2;)

    li $v0, 1
    move $a0, $t2
    syscall

# 4. QUIT THE PROGRAM "PROPERLY"

    li $v0, 10
    syscall
```

Alas, ALL of these lines of code are needed in order for us to execute our simple addition code and see the ending result inside register **\$t2** on our computer screen, i.e., on the standard output of the computer system.

So, what's going on in parts 3 and 4 in the code above? Read on...

5. Emulating a System Call

You may have learned in a prior CS course, that whenever a program needs to interact with the computer system, for example, when it needs to print something to the computer screen (otherwise known as the 'standard output' or '**std.out**'), it needs to get the attention of another (much larger) piece of software to take care of that. This other program is the **Operating System**. Whenever your program wants to print something or get a user input, for instance, it needs to issue a **system call**. This is usually not something we explicitly have to do in higher-level languages, like in C++ or Python, but it nevertheless happens inside the computer system.

In MIPS programming, and as emulated in **spim**, we will issue these system calls by doing 2 things:

- we will set up some pre-designated registers to indicate what sort of system call we want to do, and then,
- we will tell the Operating System to "execute" the system call that we want

For example, if we want to have the system print out the integer value inside a register, say, **\$t0**, then we need to:

- Set a *pre-designated register* with a very specific number code that means “we’re going to print out an integer value”. This register will always be register **\$v0**. The code for “we’re going to print out an integer value” is the number **1**.
- Make sure that *that* integer value that we want printed is copied over to yet another *pre-designated register*. That latter register is always going to be register **\$a0**.
- Finally, we have to execute the whole system call by issuing the instruction: **syscall**.

Therefore, this part of the assembly program will have these 3 instructions:

```
move $a0, $t0    # copy over the value that's in $t0 to $a0
li $v0, 1        # make the value in $v0 = 1
syscall          # execute the system call!
```

Like, **li**, the instruction **move**, is actually a *pseudo-instruction* (i.e., not a *core instruction*). More on that in a bit.

In lecture, we’ll learn how to issue the following common system calls:

- Print an *integer* value out to **std.out** (like the above example)
- Get an *integer* value as input from **std.in** (i.e., the user keyboard)
- Print a *string* value out to **std.out**
- Quit a program

6. More MIPS instructions with Additional Examples

We’ve learned about 2 “**core**” MIPS instructions: **addi** and **add**.

Here are some other common ones:

```
sub <register_d>, <register_s>, <register_t>
    register_d is a register name (destination)
    register_s and register_t are also register names (source and target)
```

Example 1: `sub $t2, $t1, $t0`
 # Subtract the value inside register \$t1 *from* \$t0
 # and put the result in register \$t2
 # i.e. equivalent of C++: `t2 = t1 - t0;`

```
sll <register_d>, <register_s>, shamt
```

register_d is a register name (destination)
register_s is a register name (source)
shamt is an “immediate” amount – a 16-bit signed integer value

Shifts a certain amount (**shamt**) of bits to the *left* of the value in **register_s** and places the result in a destination register (**register_d**)

Example 2: `sll $t1, $t0, 2`
 # Bit shift value in \$t0 2 places to the left
 # and put the result in \$t1

Example 3: This program gets a user-inputted value from std.in (i.e., the keyboard), multiplies it by 4, adds the result to itself, and finally, prints the resulting integer and quits.

```
.text
main:
    # Get an integer value from user, so make $v0 = 5
    li $v0, 5
    syscall

    # Your new input int is now the value in $v0
    # You can move it around and compute with it
    move $t0, $v0
    sll $t0, $t0, 2    # Multiply it by 4
    add $a0, $t0, $t0    # Add it to itself and place sum in $a0

    # print the result
    li $v0, 1
    syscall

    # Quit the program
    li $v0, 10
    syscall
```

Example 4: This program prints a string that's "pre-loaded" in computer memory (i.e., not in registers).

```
.data
    # This label is for when we want to define values in computer memory
    name: .asciiz "Hello World!\n"

.text
main:
    # Print string (not an int!!) to std.output.
    # Making $v0 = 4 tells syscall to expect a string to be printed...

    li $v0, 4

    # Since a string is an array of characters, we load the address of
    # that array into $a0 (we'll use a new pseudo-instruction here: la)

    la $a0, name
    syscall

    # End program
    li $v0, 10
    syscall
```

7. Pseudoinstructions

As mentioned earlier, *pseudoinstructions* can make it a little easier to program – and there's quite a few of them. In this course, however, we will very strictly restrict ourselves to using the following pseudo-instructions:

Load Immediate	li	<i>load a 32-bit value into a register</i>
Load Address	la	<i>load a 32-bit address from a .data label into a register</i>
Move	move	<i>copy a 32-bit value from one register to another</i>
Branch Less Than	blt	<i>branch to a .text label in the program if $R_s < R_t$</i>
Branch Greater Than	bgt	<i>branch to a .text label in the program if $R_s > R_t$</i>
Branch Less Than or Equal	ble	<i>branch to a .text label in the program if $R_s \leq R_t$</i>
Branch Greater Than or Equal	bge	<i>branch to a .text label in the program if $R_s \geq R_t$</i>

Pseudoinstructions are macros – that is, stand-ins – for collections of core instructions.

Pseudoinstructions are NOT core instructions themselves, meaning that there is NO 32-bit instruction equivalent machine language for them. Instead these macros are used in the MIPS emulator (**spim**) which automatically *assembles* them into the core instruction equivalents.

8. Branching and Program Flow

We all know what if/else statements are in programming!! So, what if we wanted to implement this C++ code:

```
if (x == 0) { cout << "x is zero"; }
```

In order to implement this code, we need 2 important things: (a) a way to **compare** numbers and **assess** a Boolean result from that comparison (i.e., a True or False) and (b) a way to conditionally execute code – that is to either execute code like the cout statement in our example, or to skip it if the Boolean assessment is False.

In the MIPS instruction set, we have 3 main instructions to help us with branch decisions (and while they're *not* the only instructions for this sort of thing, they are *foundational*, meaning that you don't *need* to use any other ones). These instructions are:

Instruction name	Mnemonic	Type	Syntax
Set-less-than	slt	R-type	slt \$rd, \$rs, \$rt
Branch-on-equal	beq	I-type	beq \$rt, \$rs, label
Branch-on-not-equal	bne	I-type	bne \$rt, \$rs, label

By way of an example, consider the following C++ program:

```
int main() {
    int a = 5, user;
    cin >> user;
    if (user >= a) {
        cout << a; }
    else {
        cout << 0; }
    return 0;
}
```

If we want to write this in MIPS assembly code, a good plan might be:

```
# 1. Make register $a0 = 5

# 2. Get user input
#    ("user" variable will end up in $v0, per syscall rules)

# 3. (using slt) set $t0 = 1 if $v0 < $a0;
#    meaning, $t0 = 0 if not (i.e., if $v0 ≥ $a0)

# 4. (using bne) if $t0 ≠ 0 (that is, $t0 = 1) then go to ELSE:

# 5. OTHERWISE:
#    print $a0 (use syscall, of course!)
#    jump to the end of program to quit

# 6. ELSE: (that is, if v0 < a0)
#    print the int value 0
#    jump to the end of program to quit
#    (this last part may not be necessary...)

# 7. END OF PROGRAM
#    quit program (use syscall)
```

Now, let's complete the actual code from this plan:

```
.text
main:
    # 1. Make register $a0 = 5
    li $a0, 5

    # 2. Get user input
    #     ("user" variable will end up in $v0)
    li $v0, 5
    syscall

    # 3. (using slt) set $t0 = 1 if $v0 < $a0;
    #       meaning, $t0 = 0 if not (i.e., if $v0 ≥ $a0)
    slt $t0, $v0, $a0

    # 4. (using bne) if $t0 ≠ 0 then go to ELSE:
    bne $t0, $zero, else

    # 5. OTHERWISE:
    #       print $a0 (use syscall, of course!)
    li $v0, 1
    syscall

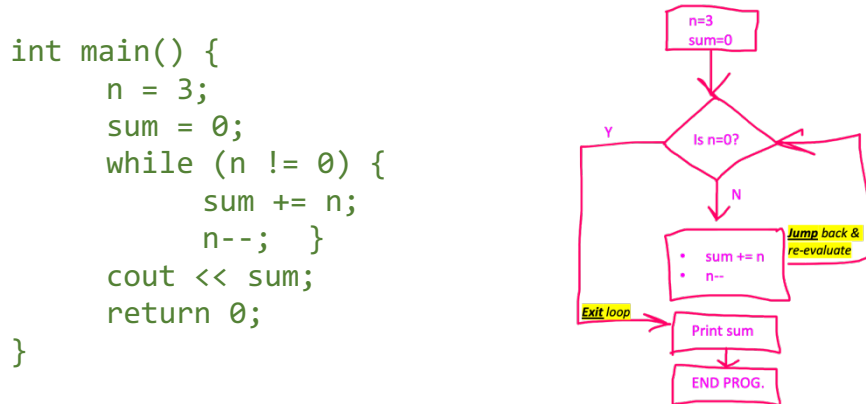
    #       jump to the end of program to quit
    j exit

    # 6. ELSE: (that is, if v0 < a0)
else:
    #       print the int value 0
    li $v0, 1
    li $a0, 0
    syscall

    #       jump to the end of program to quit
    #       (this last part may not be necessary...)
    # 7. END OF PROGRAM
exit:
    #       quit program (use syscall)
    li $v0, 10
    syscall
```

Branching in programming is also used to create **loops**: think of how a while-loop works, for example: you assess a Boolean statement and then either you run some code and then *jump back to that same assessment* (and thus decide if you should *repeat* that code), or you skip the code (and this “break” the loop).

For example, consider this C++ code and its representation as a logic-flow diagram:



The flow diagram can make it easy for us to conceive of a plan on how to program this in MIPS assembly:

```
# Let's have variables n and sum be represented by
# registers $t0 and $a0, respectively
# (this is arbitrary, but can be helpful when it
# comes time to print sum)

# 1. Set up the data registers

# 2. (using beq) check to see if n = 0 and if so, go to print_exit

# 3. OTHERWISE:
# add n to sum
# decrement n by 1
# go back to step 2 (to re-assess the need to repeat the loop)

# 4. PRINT:
# print sum and quit program
```

Now, let's complete the actual code from this plan:

```

.text
main:
    # Let's have variables n and sum be represented by
    #   registers $t0 and $a0, respectively
    #   (this is arbitrary, but can be helpful when it
    #   comes time to print sum)

    # 1. Set up the data registers
    li $t0, 3    # variable n
    li $a0, 0    # variable sum

    # 2. (using beq) check to see if n = 0 and if so, go to print_exit
loop:
    beq $t0, $zero, print_exit

    # 3. OTHERWISE:
    # add n to sum
    # decrement n by 1
    # go back to step 2 (to re-assess the need to repeat the loop)
    add $a0, $a0, $t0
    addi $t0, $t0, -1
    j loop

    # 4. PRINT:
    # print sum and quit program
print_exit:
    li $v0, 1
    syscall

    li $v0, 10
    syscall

```

We've *just* gotten started with programming in assembly! There's a lot more we can (and will) learn and do with it! For starters, in lecture, we'll also go over *nested if/else* and *nested loops* too. Of course, they use the same principles explained here – they just need a little more careful planning.

Coming up still:

1. Programming with functions
2. How memory access and addressing works
3. How we can use the stack in memory (clue: we have to do it manually...!)

Stay tuned!